

Theory of Operation

2315 cartridge archiver using Diablo 31

Summary

This project employs a Diablo Systems model 31 disk drive of either 720 KHz or 781 KHz data rate to read the contents of 2315 disk cartridges that were used on an IBM 1130 system. It then uploads the archived contents over a USB serial link to an application running on a PC, Mac or other computer that will store the disk image in the format used with the IBM 1130 simulators.

This is based around a Digilent Arty A7 FPGA board, having a Xilinx Artix A7 FPGA, plenty of dynamic RAM, buttons, switches, LEDs and quite a few input-output connections.

A custom printed circuit board is plugged into the top of the Arty A7 board. It is powered by a wall wart 9V DC supply and in turn feeds the appropriate power to the circuits on the PCB as well as to the FPGA board. The board interfaces with the voltage, impedance and current requirements of the Diablo disk drive on one side and the FPGA board on the other.

A 50 pin IDC connector on a flat ribbon cable, modified on one end to connect to the Diablo drive Winchester MPAC-42 connector, carries signals between the custom PCB and the drive.

Three 120V power supplies deliver the +15V and -15V power to the high current and low current connections on the disk drive using a Winchester MPAC-9 connector. A USB cable connects the FPGA board to the remote computer that is running the Python application supplied with the project.

Theory of Operation

2315 cartridge archiver using Diablo 31

IBM 13SD disk drive as used in the IBM 1130

The IBM 1130 System uses a single 14" platter disk cartridge (2315) inserted into an internal disk drive (13SD or Ramkit) or the optional 2310 external disk drive cabinet.

The 2315 cartridges record up to 512K words in sectors of 321 words each. The disk is organized into 203 cylinders, with an upper and a lower surface (track) per cylinder and having four sectors on each track. The platter is inside a protective plastic case.

The data is recorded at a rate of 720 kilobits per second while the disk platter spins at 1500 rpm. Thus data streams to and from the drive at 36K words per second during a read or write.

Each 16 bit word is recorded with four additional error checking bits. There is a mandatory preamble and a sync word at the start of each sector. The format for a sector is unique to the IBM 1130 and 1800 systems.

IBM software systems use only 200 cylinders for the data. The remaining four cylinders are spares. When the disk cartridge is being initially formatted, if testing determines that a spot on a cylinder cannot successfully write and read back data, the cylinder is marked as defective and one of the four spares is assigned instead.

The software using the cartridge reads the table of possible alternate cylinders when the cartridge is first mounted, then converts any movement to the defective cylinder to the assigned alternate.

The disk arm has two heads, one for each side of the 14" platter. It can move outward towards cylinder 0 or inwards towards cylinder 203, but only in steps of 1 or 2 cylinders from its current location. A microswitch confirms when the arm has reached cylinder 0 (the home cylinder).

Theory of Operation

2315 cartridge archiver using Diablo 31

Moving the arm involves setting the step size (*-10mil*) and direction (*-Reverse*) signals, then commanding a movement (*Access Go*). Feedback signals (*Access Ready*) indicate that the drive has begun the movement and then tells us when the arm is steady at the new cylinder.

Notches cut around the circumference of the disk cartridge hub define the beginning of each pie shaped slice of the platter. There are eight notches however the 1130 controller converts those eight sectors into four larger sectors which is what the software sees when it uses the drive.

Pulses arrive on the interface - a *Sector Marker* - when we reach each notch. A separate notch defines the start position. This is the *Index Marker* and lets us know that the next sector is to be considered sector 0.

The interface has a control signal called *Read Gate* which switches on the electronics to deliver pulses coming from the head as it rotates through the sector. The data on the sector is recorded serially at a rate of 720 KHz.

The drive uses a self clocking scheme since all we see from the disk surface is when the magnetic field is reversed.

Theory of Operation

2315 cartridge archiver using Diablo 31

Reading if using the 13SD drive

If we wanted to read the contents of the disk cartridge using the 13SD drive, we would access it via its simple interface. For simplicity of understanding, you can consider the 720KHz clock to be divided into two halves - high then low.

During the high portion of a clock cycle, we always record a magnetic reversal. This is the clock pulse. During the low portion of the clock cycle, we either record another magnetic reversal or we leave that spot along. If the reversal comes in the low portion of the clock, we term this a data bit value of 1. No reversal in the low portion means that we have a data bit value of 0.

The beginning of every sector consists of a long stream of 0 data bits - each having a reversal only in the high portion of the clock. The long stream allows the drive electronics to synchronize to this pattern and begin splitting the reversal pulse out over two different output lines.

The clock pulse comes out on the *Read Clock* interface line, while any pulses from the low portion of the clock cycle are emitted on the *Read Data* line. We can watch for any data pulse that occurs between successive clock pulses and consider that a data bit value of 1, otherwise when we see the next clock pulse we consider that we received a data bit value of 0.

This self-synchronizing scheme tells us how to split clock and data pulses as they are picked up by the read head, but we don't yet have a way to identify the first bit of an 1130 word. The 1130 uses 16 bit long words, thus we are extracting 16 data bit values to form a word.

Theory of Operation

2315 cartridge archiver using Diablo 31

For error checking purposes, the controller logic actually records 20 bits for each word. The last four bits are a value that brings the count of 1 data bits in the word to an even multiple of four. If the 16 bit 1130 word was already at an even multiple of four - for example values x0000, x0238 and xffff have 0, four and sixteen 1 bits thus all are an even multiple of four. Our check bits would be 0000

The check bits are emitted to bring the count of 1 bits up to 0 modulo 4. For example, if we had the word x8000 with a single 1 bit, then the first three check bits are 1 and the last is 0, bringing the count up to 4.

To know when we are beginning a word recorded as 20 bits, we need to delineate when the first data bit would occur. that could be a 0 data value or a 1, so we have a special word value which is the first word recorded after the long string of 0 data bits. This word is x0001 with check bits 1110.

The logic reading from the drive turns on the *Read Gate* when we reach the sector, waits for a while to let the string of 0 data bits synchronize the clock/data splitter, then we watch for a data word that is the bit pattern 000000000000000011110 and the very next data bit will be the first bit of the first word of the sector.

The sector has up to 321 words of data recorded on it - 6,420 clock pulses, which we can now separate into words because we are synchronized. Typically we turn off the *Read Gate* after we read the last word. The 1130 can request from 1 to 321 words to be read, thus that count is used to decide when to turn off the *Read Gate*. For most purposes the 1130 reads all 321 words of a sector.

Theory of Operation

2315 cartridge archiver using Diablo 31

We have a *Head Select* signal that tells the drive to read or write from either the top head or the bottom head. Each head rotates past four sectors during one revolution of the disk drive. By reading four sectors on one side, switching the head and then reading another four sectors, we can read all 2,568 words of that cylinder.

Setting the direction and step size first, we can ask the disk drive to move the arm its 1 or 2 cylinders. When the feedback tells us we are at our new cylinder location, we can read or write the sectors at that arm position.

Writing new contents into a sector involves using the *Write Gate*, *Erase Gate* and the *Write Clock & Data* signal lines. When the *Sector Mark* indicates we are at the intended sector position, we turn on the *Write* and *Erase Gate* lines. We begin emitting a string of pulses on the *Write Clock & Data* signal line at 1,440 KHz rate. We alternate a clock pulse with either a data pulse or no pulse, so that we are generating the self clocking stream of magnetic reversals on the disk.

We begin by sending out only pulses for the clock portion, thus we write a long string of 0 data bits. When ready, we write a stream of pulses on *Write Clock & Data* - 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 1 1 1 1 1 1 0 - which represents data bit values 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 or our sync pattern word. We then write out the data words, first the sixteen data bit values as either 1 1 or 1 0 depending on the bit value, followed by the check bit pattern to bring our count of 1 value data bits to an even multiple of four.

When we have written the last word we want on that sector, we turn off the *Write Gate* and the *Erase Gate* control lines. Typically, all 321 words are written to a sector by 1130 software.

Theory of Operation

2315 cartridge archiver using Diablo 31

The only other signals on the interface are *File Ready* - the drive is ready to read/write/seek - a *720KHz clock* which is generated inside the disk drive, and a *90 second delay* signal that lets us know that the drive started but has not yet allowed enough time to purge any dust or contaminants from the surface of the disk. After 90 seconds the drive lowers the heads onto the disk surface and they ride at cylinder 0 waiting for commands over the interface.

Theory of Operation

2315 cartridge archiver using Diablo 31

Diablo Systems model 31 disk drive

IBM offered licenses to other manufacturers who produced disk drives and disk cartridges based on the 13SD drive and 2315 cartridge designs. Several firms made disk cartridges and several developed disk drives for their minicomputer systems under license.

The companies that licensed the disk technology were free to improve on the 13SD drive and most did. The IBM drive used a mechanical ratchet system for arm movement between cylinders, thus is moved in steps of 1 or 2 cylinders at a time. All the other manufacturers designed a servo system that supports a single move of any number of cylinders.

The rate at which data is clocked onto the platter - 720KHz in the case of the IBM drive - was implemented at higher rates of 781, 1440 and 1562 KHz by those makers. The two higher rates were labeled High Density.

Diablo Systems was one of the companies that built drives which were used by computer makers such as Xerox, Varian, and Interdata. Other companies such as DEC and HP licensed the technology and made their own drives.

The Diablo Model 31 comes in many flavors, described by the Options configured on the drive. Option 001 was for standard density 720KHz operation, also called IBM compatible.

Model 31 could operate at any of four data rates (720KHz, 781KHz, 1440 KHz and 1562KHz) and make use of cartridges with 8, 12, 16, 24 or 32 sectors per revolution. Xerox PARC (Palo Alto Research Center) created their groundbreaking Alto computers using a high density model 31 drive.

Theory of Operation

2315 cartridge archiver using Diablo 31

Using Diablo drive to read 1130 cartridges

Model 31 drives can be used with this project to read 2315 cartridges that were written on IBM 1130 systems, but must match the specifications of the 13SD drive. Thus, they must use 8 sector cartridges and have a data rate of 720KHz. If so, the Diablo drive and the 1130 can read data written on the other system, giving full interchangeability.

We provide a way to modify a Model 31 from Option 002 - 781KHz data rate - so that it can operate at the 720KHz rate. The higher density options use a different kind of read/write head and are incapable of interchanging data with the 13SD drive.

The interface of the Diablo Model 31 is more complex than the 13SD, plus there are many options to suit the many manufacturers who used these drives with their computers. A key difference is in how the disk arm is moved to a desired cylinder.

The Diablo interface has a set of eight signal lines (*Track*) that specify an absolute cylinder location. It does not have a direction or a step size. Instead, the interface is given the destination cylinder and then the *Strobe* signal is activated to request the movement. No *home* cylinder microswitch signal is included because the drive always knows where it is.

The feedback mechanism for a seek is a bit more complex - with timing requirements for when the *Ready to Seek*, *Read or Write* signal must drop and when it turns back on at the end of a movement. An *Address Acknowledgement* signal is sent to inform the computer that the drive has accepted the movement request.

Failure of the seek mechanism is indicated by *Seek Incomplete*. Requests to move to a cylinder beyond 203 can trigger a *Logical Address Interlock* signal if that option is included on the drive.

Theory of Operation

2315 cartridge archiver using Diablo 31

Sector Mark and *Index Mark* pulses are generated just like with the 13SD. However, this does require that the drive is configured for 8 sectors and the 2315 Disk Cartridge is 8 notches cut around the hub. Diablo also supports 12, 16, 24 and 32 sector formats which are incompatible with IBM.

Besides the use of the Sector Mark and Index Mark pulses, the interface offers a five bit Sector Number field that reports the upcoming sector number directly. There is also a *Pseudo-Sector Marks* which emits pulses at a 50KHz rate so that a computer interface can calculate its own logical sector lengths, disregarding the physical notches.

Reading and writing are controlled in very similar ways to the IBM 1130. *Read Gate* and *Write Gate* are used the same way. *Erase Gate* is normally activated automatically when *Write Gate* is asserted, but an option exists to control them individually. *Head Select* is used to switch between the upper and lower sides of the platter.

The drive uses the same self-clocking scheme as the 13SD, thus pulses come out on the *Read Clock* and *Read Data* signals just as they do with IBM drives. Writing involves sending the mixed clock and data pulses on *Write Clock & Data*, again just like the 1130.

There are some signal options such as *Write Protect* which blocks the drive from erasing or writing, *Write Check* indicates a failure of the writing electronics and locks up the drive. *Restore* which brings the arm back to Cylinder 0 and resets any drive error condition.

Finally, the Diablo drives can be daisy-chained off a single disk controller, with up to four drives connected. The interface has signals *Select 1*, *Select 2*, *Select 3* and *Select 4* to allow the controller to choose which drive should respond to its commands. *Interrupt* and *attention* signals are also options but only rarely found on the drives.

Theory of Operation

2315 cartridge archiver using Diablo 31

Logic in the FPGA

Besides offering a boatload of logic elements, the Arty A7 board has 256MB of DDR3 DRAM which is very handy to store the contents of a disk cartridge which itself holds only 1MB of data. The onboard clock crystal runs at 100 MHz, far faster than any events happening on the disk drive (or even in the IBM 1130).

I set up a clock generator to produce the faster clocks required by the DDR3 memory controller intellectual property. In addition to the memory interface IP and the clock IP, I defined two FIFO queues using IP that exploits the native FIFO hardware on the FPGA chip. This is because the memory interface runs at a different clock rate than the general 100MHz logic of my design, thus the FIFO allows the data to cross clock domains safely. The memory interface produces the `ui_clk` clock used to drive the memory interface.

All external signals run through a chain of four flipflops to protect against metastable states since the timing of signal changes externally is unrelated to the clock edges inside the FPGA. I then pass the through a Schmitt Trigger module I wrote so that brief waggles in signals don't cause the logic to switch states at the wrong time.

Everything I coded is in Verilog, although the IP and some borrowed modules may be in VHDL. In some ways, it is influenced by the RK05 Emulator code. I split the design into functional modules such as read a sector, seek to a cylinder, track the sector position, and then higher level modules that drive the process of archiving by moving serially through all 1,634 sectors across 203 cylindere.

Theory of Operation

2315 cartridge archiver using Diablo 31

The top level module just routes signals to the various functional modules and externally. to the interface board. It displays status on the eight LEDs and uses a slide switch and a button to request the archive step and then the upload across the the USB serial link.

Archiver module

The archiver logic starts off at cylinder 0, upper head, sector 0 and word 0. When the button is pressed to start the state machine, it advances through the states until it has completed reading all 1,634 sectors and 521,304 words and storing them into the RAM.

It loops waiting for the sector number to change to the target number, then kicks off the bus_read_drive module to read the sector. When that module responds with a completion signal, the state machine advances.

The next step advances through sectors, from 0 to 1 to 2 to 3, then returns to wait for that sector to arrive. When sector 3 has finished its read, the next step switches the head to the other platter and resets the sector target to 0. It then returns to wait for the sector.

After we finish sector 3 of the second head, we move to a step that advances the cylinder by 1, resets the head and the sector before returning to wait for the sector.

When we have finished the last sector, instead of going back like we did for the first 202 cylinders, we turn on a feedback signal indicating we are done with archiving and then reset to the idle state until we get a new request for archiving.

Theory of Operation

2315 cartridge archiver using Diablo 31

Sector and Index module

The module watches for the leading edge of the *Sector Marker* and *Index Marker* pulses to track the position of the platter as it rotates. It must swallow every other *Sector Marker* because that is how the 1130 controller logic converts the eight physical sectors into four logical one. The module emits the upcoming sector number to the other modules and a pulse when the *Sector Marker* leading edge arrives to start that sector.

Seek to Cylinder module

The state machine matches the timing spec of the Diablo model 31 drive. When a *Strobe* request arrives, the state machine first emits the new *Track* number to the drive. It then waits for *Addr_Ack* to be asserted by the drive, then loops for 3 microseconds. After that time, it resets the *Track* number. The state machine then drops the *Strobe* signal and waits for the drive's *Ready_Seek* to go low, then returns to the idle state.

Theory of Operation

2315 cartridge archiver using Diablo 31

Bus_Read_drive module

When a *requestread* signal arrives to read a sector and store the words into the DRAM, the state machine waits for the leading edge *clkenable_sector* pulse from the *Sector and Index* module that indicates we reached the start of a sector.

The first state raises the *Read_Gate* signal and sets up to wait through reading 100 bit values of 1 from either the *Read Clock* or the *Read Data* lines. The preamble should be a long string of clock pulses with no data pulses which allows the drive to properly split pulses as clock or data.

After we see 100 pulses, it is time to move to the state that waits until it sees the special sync word pattern - 0b000000000000000011110 which is a word of single trailing bit of 1 and the correct check bits 0b1110 to reach a multiple of four bits with the value 1.

It is detected when the first 1 value arrives on *Read_Data*, then it advances to verify that the next four bits are 1 1 1 0. to confirm that it is the sync word. It sets up a word count of 321 words and advances to accumulate the 20 bits of the word and its check bits. If the sync word wasn't validated, the *syncerror* state is recorded.

The logic watches the falling edge of the *Read Clock* pulse then if it sees a *Read Data* pulse before the rising edge of the next *Read Clock*, we set the *got_one* signal.

We clock in the *got_one* bit value into a shift register on every rising edge of *Read Clock*. We also maintain a count of how many 1 bits were detected. After 16 clock pulses we stop shifting the bits in but we continue counting 1 bits for the next four clock pulses, which are the check bits. At the end of the word, we detect an error if the counter doesn't reach 0 modulo 4.

Theory of Operation

2315 cartridge archiver using Diablo 31

At the end of the word, it bumps the word number then at the falling edge of the *Read Clock* pulse, it raises the *grabword* signal and sets up the data from the shift register for the write to the DRAM.

We go back to read the next word until we complete all 321 words then we save any errors before finishing.

If we saw an error on any word, because the four check bits didn't advance the ones counter to a multiple of four, we store that condition in word 322. We also store the state of the *syncerror* state. That allows us to keep track of whether any sectors had errors while being archived.

The state machine sits at the last step, raising *readdone* to the *archiver module* and waiting until the archiver drops the *requestread* signal before we go back to idle.

Theory of Operation

2315 cartridge archiver using Diablo 31

dram_controller module

This module is responsible for both reading and writing from the DRAM. The address in DRAM is formed from the cylinder, head, sector and word number. Requests *grabword* and *echoword* are the triggers asking this module to write to DRAM or to read from DRAM respectively.

This module works in two different clock domains, the 100 MHz main clock and the *ui_clk* clock produced by the memory interface generator IP. Any request to read or write memory must be done with the *ui_clk* clock.

I created two FIFOs to pass information between the two clock domains. The write FIFO takes words from the *archiver module*, in the 100 MHz clock domain, and passes them to the other side of the FIFO which operates in the *ui_clk* domain. The read FIFO accepts data from the DRAM in the *ui_clk* domain and the data pops out on the other side in the 100 MHz domain to be returned to the *uploader module*.

The flow when writing data is to have the *grabword* trigger the routine to push the word into the write-FIFO and then the *ui_clk* pump sees that valid data has arrived and uses it to drive a write into memory. When the write is confirmed by the memory interface, the *ui_clk* pump goes back to idle.

The flow when pulling data is to have the *echoword* trigger the routine to push a request into the write-FIFO. The *ui_clk* pump grabs the request, drives a read request to the memory interface and puts the output into the read-FIFO where it can be returned. It sets *dataready* on and waits until the *echoword* request is dropped.

Theory of Operation

2315 cartridge archiver using Diablo 31

Uploader module

This state machine is a simple loop, advancing word within sector within head within cylinder. On each new sector, a line is printed over the serial interface giving the cylinder, head and sector number. It then writes one line for each of the 321 words in the sector. A final line is printed with the error state of the sector.

For each word, we have generated the DRAM address using the cylinder, head, sector and word number. It raises *echoword* to trigger the dram_controller to read the word. When the memory content is returned, the *dataready* signal goes high and we drop our *echoword* request and move on to the next step.

The state machine formats the ASCII text for each line to be written over the serial link. We look at the value of the Cylinder, Head and Sector field and convert those into the ASCII codes for the decimal digits. These are inserted into the buffer holding the sector address message.

This module leverages modules written by William Carter - UART_DEBUGGER2 and YASUTX - which send out a message stream such as the header we just formatted as well as a function to write a data word with ASCII characters for the hex value. The serial line operates at 115,200 baud, 1 start bit, 1 stop bit, 8 data bits and no parity.

We also format a message with codes for whether we saw a sync error or an ECC error while archiving this sector. It is transmitted after we finished the 321 words of the sector.

When we have written the error state message for the 321st word of sector 3 of head 1 of cylinder 203, turn on the *dumpdone* signal to tell the too level module that the upload is complete.

Theory of Operation

2315 cartridge archiver using Diablo 31

Handling inputs from the disk drive to the FPGA

The disk drive terminated the disk end of the cable with a 120 ohm resistor to +5V and a 300 ohm resistor to ground. The signal line will sit about 3.6 volts when the signal is not asserted and pulled down to ground when the asserted.

On my PCB, the other end of the signal has the same 120 ohm and 300 ohm resistors. As long as the drive output gate pulls enough current to ground, the signal wire on my end will be below the logic low threshold. The wire is connected to a 74LVC125 buffer chip. The buffer is powered by a 3.3V rail produced on the PCB. The input of the buffer is tolerant of up to 5.5V thus it is safe to connect the drive signal to the chip. The chip output is 0 or 3.3V which is connected to the FPGA input pin.

Handling outputs to the disk drive from the FPGA

The disk drive end of the command signals are terminated with a 220 ohm resistor up to +5V and a 510 ohm resistor down to ground, yielding a 3.5V voltage when I am not asserting the signal. I have a 10K pullup resistor on my end of the wire and a fast MMBT2369 transistor that will pull the line down to ground to assert it.

The FPGA output pin produces a 3.3V signal to assert the output line, driving about 9ma through a 330 ohm base resistor to cause the transistor to pull the line to ground. When I drop the FPGA output to 0V, the transistor is cut off and the weak 10K resistor pulls the line up towards a 5V rail I also produce on the circuit board, but the terminator resistors on the disk drive end essentially pull the voltage to 3.5V.

Theory of Operation

2315 cartridge archiver using Diablo 31

Power circuits on the board

A 9V wall wart power supply feeds the circuit board. Two voltage regulators on the board are used to generate a +5V and a +3.3V rail. The +5V also feeds the Arty A7 FPGA board.

Power for the disk drive

The disk drive is powered by +15VDC and -15VDC. That operates not only the electronics in the drive, but also the motors and servo to move the disk arm. The drive consumes an average of 4A from each rail when operating and has burst requirements to about 8A.

A dedicated power supply fed from the AC mains is used to generate the high current to the drive, one each for +15 and -15. The logic and analog circuits in the drive also need +15V and -15V but they should be fed by independent power supplies to avoid noise and impedance effects from the high current devices like the motor. A third supply hooked to the AC mains is a lower current dual supply, delivering 2A of +15 and 2A of -15 to the drive.